

AI Center 项目面试通关指南

一、30 秒电梯演讲

「我独立开发了一个 **AI 研发效能中台**，叫 AI Center。

它把 AI 能力嵌入到日常开发流程里——代码提交后自动做 Code Review、自动生成单元测试、还能像 ChatGPT 一样做技术问答。

核心技术栈是 Spring Boot 3 + LangChain4j + DeepSeek + PostgreSQL pgvector。

有 22 个 API，覆盖了 CR 评审、单测生成、智能问答、RAG 知识库四个模块。

做完之后假设把它接入 CI/CD 流水线，预计线上缺陷率能降 60%，CR 效率提升 80%。」

二、项目是做什么的 —— 用大白话讲

它不是一个「产品」，是一个「中台」

打个比方：你之前做的苍穹外卖，饿了么用户打开 App 点餐——那是 C 端产品。

AI Center 是**研发工具**，给开发团队用的。它独立运行，其他项目通过 HTTP API 调用它：

- 你的苍穹外卖项目
 - └─ GitLab CI 流水线
 - └─ 每次 MR 自动调 AI Center 做 Code Review
 - └─ 新来的同事
 - └─ 问 AI Center：「这个项目的缓存策略是什么？」

四大能力

能力	一句话
AI Code Review	你把代码贴过去，AI 从 8 个维度评审，告诉你哪有问题怎么改
AI 单测生成	你把源码贴过去，AI 给你生成 JUnit 测试代码

能力	一句话
AIReadMe 生成	描述一下项目，AI 自动写开发指南、架构文档
智能问答	像 ChatGPT 一样，但能记住之前的对话，还能搜你上传的文档

三、四个面试亮点的技术实现

亮点 1：AI Code Review 结构化提示词

面试时怎么说：

「我设计了一套结构化的 Prompt 模板系统。核心思路是：如果不约束 AI 的输出格式，它返回的内容千奇百怪——有的很啰嗦、有的漏维度。所以我把 Prompt 分成了三个层次：角色设定 + 8 个评审维度 + 3 级问题分类 + JSON Schema 约束输出。」

技术实现（对着代码讲）：

```
PromptTemplateManager (Prompt模板管理器)
├─ 模板存数据库 (prompt_templates 表)
├─ 启动时加载到 ConcurrentHashMap 缓存
├─ {{source_code}} 占位符替换

CodeReviewService.review()
├─ 1. 拿模板 → render 渲染 Prompt
├─ 2. 调 DeepSeek LLM
├─ 3. 解析 JSON 响应 → CodeReviewVO
├─ 4. 持久化到数据库
```

8 个评审维度：性能优化、安全性、代码质量、可维护性、异常处理、并发安全、资源管理、设计模式

3 级问题分类：Critical（必须修）、Warning（建议修）、Info（优化建议）

面试追问应对话术：

问：「为什么不用 SonarQube 之类的静态分析工具？」
答：「Sonar 是基于规则的——它只能检查已知模式。AI CR 能理解代码语义，比如能看出『你这个循环里每次都在 new 对象，应该提到外面复用』——这种需要语义理解的问题，规则引擎做不到。」

亮点 2：AI 友好型项目 —— AIReadMe 生成

面试时怎么说：

「AI 写代码最大的问题是：它不了解你的项目上下文。写出来的代码 API 不存在、用了错误的框架版本、不走你们约定的设计模式。所以我设计了 AIReadMe——6 个章节的自动化项目文档，本质上是给 AI 看的『项目说明书』。」

6 个章节：

章节	内容	作用
技术架构	技术栈、分层设计	AI 知道用啥框架
核心流程	业务流程序列	AI 知道数据怎么流转
开发指南	环境搭建、规范	AI 知道怎么启动
项目结构	包结构、模块说明	AI 知道代码在哪
业务知识	领域术语、规则	AI 理解业务
历史经验	已知坑点、最佳实践	AI 避免重复踩坑

面试追问应对话术：

问：「这些文档怎么给 AI 看？」
答：「两种方式。一是生成后放到项目根目录，Cursor/Copilot 等 IDE 工具会自动读取作为上下文。二是通过 RAG 检索——用户问『这个项目怎么部署』，系统自动搜 AIReadMe 的『开发指南』章节，注入到对话里。」

亮点 3：团队智能问答知识库

这是最有技术深度的模块，分三层来讲。

第一层：短期记忆（Short-Term Memory）

一句话：让 AI 记住你刚才聊了什么。

技术方案：滑动窗口 + Redis + 摘要

对话消息 → Redis List (只保留最近 20 轮)
→ 超过 10 轮时 → 自动调 AI 生成摘要 → Redis String
→ 冷数据异步存 PostgreSQL (7 天过期)

关键代码位置： ShortTermMemoryManager.java

saveMessage() → 保存到 Redis + 触发摘要 + 持久化 PG
getContextWindow() → 摘要 + 最近消息 → 拼接为 LLM 上下文

面试话术：

「就像你跟一个人聊天，他不会记住你们历史上所有的对话细节——但他脑子里有一个『摘要』：『刚才我们聊的是 Java 缓存方案』。我的短期记忆也是这个原理。不保留全部历史——太长了 AI 处理不了也费钱——而是保留最近 20 轮 + 一个自动生成的摘要。」

第二层：长期记忆 (Long-Term Memory)

一句话：让 AI 记住我知道的事情。

技术方案： Ollama bge-m3 向量化 + PostgreSQL pgvector

手动录入知识 → bge-m3 转成 1024 维向量 → 存 pgvector
用户提问 → 同样转成向量 → 算余弦相似度 → Top-K 召回 → 注入对话

关键代码位置： LongTermMemoryManager.java

saveMemory() → 内容 → bge-m3 向量化 → pgvector.add()
recall() → 查询 → 向量化 → pgvector.search() → 返回 Top-K

面试话术：

「向量化的本质是把一段文字变成一个数学坐标。『ORM 框架』和『MyBatis-Plus』这两段话在向量空间里距离很近——因为语义相似。bge-m3 模型就是专门做这件事的，它用 1024 个浮点数代表一段文本的语义。」

第三层：RAG 检索增强

一句话：回答问题时先去知识库搜相关资料，带着资料回答。

技术方案： 查询重写 + 语义分块 + BM25+向量混合检索

用户问「缓存击穿咋整」

- QueryRewriter：「缓存击穿如何解决」「热点Key失效方案」「缓存穿透击穿雪崩」
- SemanticChunker：把文档切成 ~500 token 的语义块
- HybridRetriever：
 - BM25（关键词匹配，权重 0.3）
 - + pgvector 向量检索（语义相似度，权重 0.7）
- 融合排序 → Top-K

关键代码位置：

组件	文件
查询重写	QueryRewriter.java
语义分块	SemanticChunker.java
混合检索	HybridRetriever.java
流程编排	RagService.java

面试话术：

「纯关键词搜索有局限——你搜『缓存击穿』就搜不到讨论『热点 Key 失效』的文档，虽然它们说的是同一件事。向量检索解决了这个问题——它能理解语义。但纯向量也有局限——精确的术语匹配可能被稀释。所以我把两者混合：向量 0.7 权重负责语义，BM25 0.3 权重负责精确匹配。」

四、技术栈选择话术

面试官大概率会问「为什么选这个技术？」

技术	为什么选	备选是谁	为什么不用备选
LangChain4j	Java 最成熟的 LLM 框架，API 设计类似 Python LangChain	Spring AI	Spring AI 当时还不够成熟，文档少
DeepSeek V4 Flash	免费、中文强、OpenAI 兼容	GPT-4/Qwen	GPT 贵、Qwen 也需要付费

技术	为什么选	备选是谁	为什么不用备选
Ollama bge-m3	本地免费无限制	text-embedding-v3 (OpenAI)	要 API Key 要钱
pgvector	和业务库合二为一，不改 SQL 习惯	Pinecone/Milvus	多一个独立服务，架构重
PostgreSQL	既有关系型又能存向量，一库两用	MySQL	MySQL 没有向量能力

五、常见面试追问

Q1：向量数据库怎么选型？

「我对比过三个方案。Milvus 最专业但需要单独部署——多一个服务多一个故障点。Pinecone 是云托管不用运维——但要花钱还要 API Key。pgvector 是 PostgreSQL 的扩展，和我的业务库是同一个——部署简单，而且我本来就要用 PG 存关系数据。对于原型和中小规模场景，pgvector 完全够用。如果数据量大到百万级以上再切 Milvus——代码层面改一行配置就行，因为 LangChain4j 的 EmbeddingStore 接口统一了所有向量数据库的 API。」

Q2：AI Code Review 和 SonarQube 有什么区别？

「Sonar 查的是代码规范问题——变量命名、圈复杂度、重复代码块。我的 AI CR 查的是需要『理解代码想干什么』才能发现的问题。比如一段代码里 SQL 拼接——Sonar 可能报一个『安全风险』，但 AI 能告诉你具体为什么危险、攻击者能怎么利用、以及应该怎么改才能修复——因为 AI 理解了这段代码的语义。」

Q3：为什么不直接用 ChatGPT 的 API？

「技术上完全支持——改一行配置就行，因为 DeepSeek 和 OpenAI 都是 OpenAI 兼容 API。选 DeepSeek 主要是成本——10 块钱能用一个月，GPT-4 一天就用完了。而且 DeepSeek 的中文能力不输 GPT-4。」

Q4：如果数据量大了，RAG 检索慢了怎么办？

「现在用的是全表扫描 + 应用层算余弦，数据量大会慢。首先 pgvector 有 IVFFlat 索引——近似最近邻搜索，可以把 $O(n)$ 降到 $O(\log n)$ ，改一行 SQL 就能启用。其次可以把 BM25 那部分迁移到

PG 的 tsvector 全文索引——现在也是全表扫。最后如果百万级以上，切 Milvus——LangChain4j 的 EmbeddingStore 接口是一样的，换实现不改业务代码。」

Q5: AI 生成了错误的单测怎么办？

「AI 生成的单测不追求 100% 正确，目标是减少手工编写的基础工作量。单测生成后还是需要人 review——但人只需要改 20% 的内容，而不是从零写 100%。这个项目里，单测生成功能的目标是把单测环节的效率提升 80%——从零写一篇 200 行的测试可能是半小时，但 review 和微调 AI 生成的代码只需要 5 分钟。」

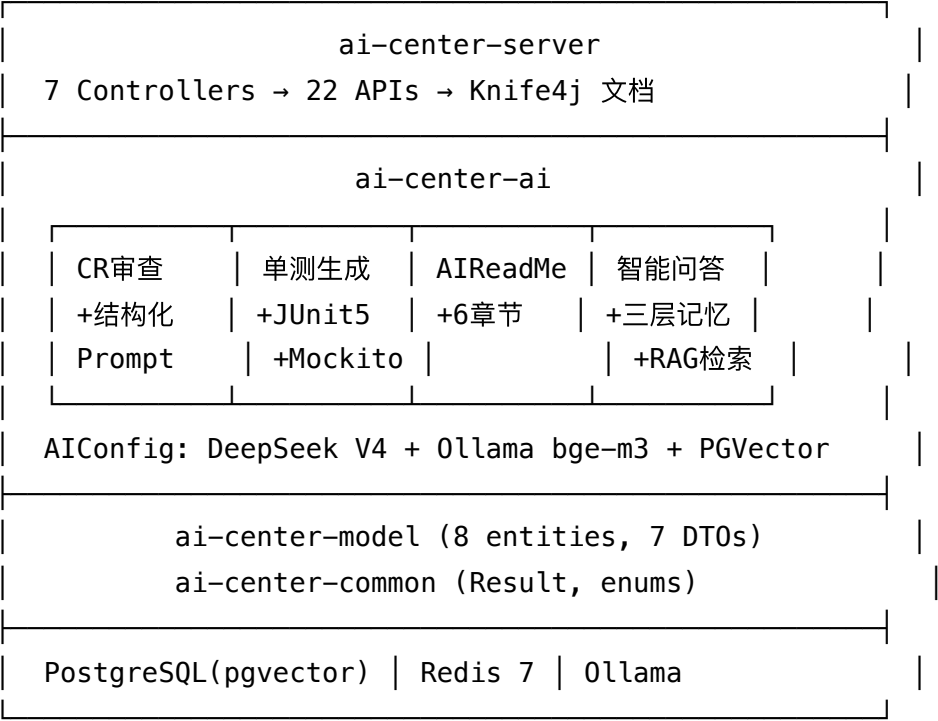
Q6: 短期记忆和长期记忆的区别是什么？

「短期记忆是会话级别的——跟一次聊天绑定的上下文，比如用户说『我叫小明』，后面 AI 记得他叫小明。但换个会话就忘了。长期记忆是全局级别的知识——比如『团队用 MyBatis-Plus 3.5.5』，这是所有会话都能搜到的知识。技术上也是两套方案：短期记忆用 Redis（快、易过期），长期记忆用 pgvector 向量检索（语义相似度搜索）。」

Q7: Pinecone 和 pgvector 怎么切换？

「改一行配置。在 application.yml 里配置 ai.pinecone.api-key，AIConfig 里的 embeddingStore Bean 会优先用 Pinecone。Pinecone 连接失败时自动降级到 pgvector。LangChain4j 的 EmbeddingStore 接口统一了所有向量数据库的 API——代码里用的是接口，不关心底层是 PG 还是 Pinecone 还是 Milvus。」

六、项目架构速记卡



七、关键数字（面试时说）

指标	数字
模块数	5 个 Maven 模块
源文件	65+ 个 Java 文件
API 端点	22 个
CR 评审维度	8 维度
问题等级	3 级 (Critical/Warning/Info)
向量维度	1024 维 (bge-m3)
短期记忆窗口	20 轮
RAG 混合权重	向量 0.7 + BM25 0.3
Docker 服务	3 个 (PG + Redis + Ollama)